Machine Learning for Kira IBP Reduction

Seed selection and elimination ordering

October 1, 2026

Abstract

Feynman integral calculations are an important component of multiloop amplitude calculations in quantum field theory. In Quantum Chromodynamics (QCD), multiloop amplitudes and their associated Feynman integrals arise in higher-order perturbative calculations and can become increasingly complex at higher loop orders. Recent progress has automated these calculations to some extent using the framework Kira, which organizes target integrals into integral families and sectors, generate algebraic relations among the integrals by using integration by parts (IBP) identities, and reduce the resulting system to expressions in terms of a finite set of master integrals. We aim to explore the application of System 1 machine learning models to accelerate the IBP reduction process by investigating if lightweight learned models can improve seed selection. These models can predict the most useful integrals from which to generate IBP relations while minimizing redundant calculations. We will also explore System 1 models for learning efficient reduction and elimination sequences to minimize the number of algebraic operations required to express target integrals in terms of master integrals. Because the machine learning models will serve as decision-makers while the underlying IBP reduction and verification remain exact, these models can improve computational efficiency without compromising the correctness of the resulting reductions.

1 Introduction

Feynman integrals arise in higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes. In QCD, these corrections involve quark and gluon loops and contribute to predictions for cross sections and decay rates. Calculating a loop contribution requires integration over the independent internal momenta that remain after imposing momentum conservation [1].

The scalar integrals that appear in an amplitude have the general form

$$\mathcal{J} = \left(\prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}\right) \frac{\mathcal{N}(\boldsymbol{\ell}, \boldsymbol{p})}{\prod_{i=1}^{n_{\mathrm{phys}}} D_i^{a_i}}, \qquad D_i = q_i^2 - m_i^2 + \mathrm{i}0. \tag{1}$$

Here L is the number of loops and $\boldsymbol{\ell} = (\ell_1, \ldots, \ell_L)$ is the list of independent loop momenta over which we integrate. The number of independent external momenta is E, and we keep these momenta, $\boldsymbol{p} = (p_1, \ldots, p_E)$, fixed during integration. The numerator $\mathcal{N}$ is a polynomial obtained from the spin and tensor algebra; we leave its mass dependence implicit in the notation. The number of physical denominators is n_{phys}, and each D_i corresponds to an internal particle line with momentum q_i and mass m_i. The positive integer a_i is the power of D_i in the denominator, and q_i is a fixed linear combination of loop and external momenta, with $q_i^2 = q_i \cdot q_i$ denoting the Lorentz scalar product.

The integration dimension is $d = 4 - 2\epsilon$, where ϵ is the regulator used to define divergent integrals, so the result depends on ϵ as well as the external momenta and masses. The factor $1/(\mathrm{i}\pi^{d/2})$ is our normalization convention for each integration measure, with $\mathrm{i}^2 = -1$. In each denominator, i0 is an infinitesimal positive imaginary term that specifies how to integrate past a propagator pole, where $q_i^2 = m_i^2$.

Higher loop orders produce collections of integrals with related numerators and denominators and algebraic identities can be used to express them in terms of an independent set of master integrals. Kira automates this reduction [2], after which the calculation requires the values of the master integrals. To explain how the reduction works and where machine learning (ML) could speed it up, we first define the notation used in this paper, then derive the identities and describe how Kira solves the resulting equations.

2 The mathematical form of Feynman integrals

To label the integrals related to Eq. (1), we specify their momentum dependence and write the numerator and denominator powers as a vector of integers. This vector identifies each integral as an unknown in the reduction equations.

2.1 Loop momenta, propagators and regularization

After choosing how the loop and external momenta flow through the internal lines, we write each physical denominator as

$$D_i = q_i^2 - m_i^2 + \mathrm{i}0, \qquad q_i = \sum_{k=1}^{L} \alpha_{ik} \ell_k + \sum_{\beta=1}^{E} \eta_{i\beta} p_\beta. \tag{2}$$

The constants α_{ik} and $\eta_{i\beta}$ specify which loop and external momenta make up q_i, with indices k, j for loop momenta and β, γ for external momenta. Squaring q_i gives the scalar products $\ell_k \cdot \ell_j$, $\ell_k \cdot p_\beta$ and $p_\beta \cdot p_\gamma$, and those containing loop momenta vary during integration. The

vector $\boldsymbol{x}$ contains the independent external scalar products and squared masses, which are the kinematic parameters we keep fixed during integration.

Loop integrals can diverge at large momenta or when massless momenta become small or parallel. Dimensional regularization defines these integrals at values of d for which they converge and extends the result by analytic continuation to $d = 4 - 2\epsilon$, using additional regulators where necessary. In an expansion around $\epsilon = 0$, the divergences appear as negative powers of ϵ, which renormalization and infrared cancellation in the observable address later in the calculation [1]. Having specified the regulated measure and momentum dependence, we can now use the powers of the numerator and denominator factors to label individual integrals.

2.2 A common notation for an integral family

We add auxiliary expressions to the physical denominators so that we can write every required loop-momentum scalar product in terms of the list $D_1, \ldots, D_N$ and the fixed parameters $\boldsymbol{x}$. Here N is the number of expressions in the list, and the auxiliary expressions allow us to represent numerator factors. Writing the numerator in terms of this list expresses $\mathcal{J}$ as a linear combination of integrals of the form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \prod_{i=1}^{N} D_i^{-a_i}, \qquad \int_\ell \equiv \prod_{k=1}^{L} \int \frac{\mathrm{d}^d \ell_k}{\mathrm{i}\pi^{d/2}}, \qquad \boldsymbol{a} = (a_1, \ldots, a_N) \in \mathbb{Z}^N. \tag{3}$$

Here $\int_\ell$ is shorthand for the loop integration measure and $\boldsymbol{a}$ is the vector of N integer powers, with $\mathbb{Z}$ denoting the set of integers. These powers can be positive, zero or negative, so we can write numerator and denominator factors in the equivalent form

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \int_\ell \frac{\prod_{i:a_i<0} D_i^{-a_i}}{\prod_{i:a_i>0} D_i^{a_i}}. \tag{4}$$

A positive index places $D_i^{a_i}$ in the denominator, a negative index places $D_i^{-a_i}$ in the numerator, and a zero index removes the factor, with an empty product equal to one. Auxiliary expressions used only as numerators therefore have nonpositive indices. An irreducible scalar product is a numerator scalar product that cannot be written as a linear combination of the physical denominators and terms depending only on $\boldsymbol{x}$ [3].

An integral family contains the integrals obtained by varying $\boldsymbol{a}$ for a fixed list $D_1, \ldots, D_N$ and fixed kinematics. Within this family, the target list specifies the integrals that the amplitude requires, and grouping family members by their positive indices organizes them into sectors.

2.3 Sectors and the organization of related integrals

A sector is a set of integrals with the same positive-index positions, so they contain the same denominators. We assign a label to each sector using

$$\theta_i(\boldsymbol{a}) = \begin{cases} 1, & a_i > 0, \\ 0, & a_i \leq 0, \end{cases} \qquad S(\boldsymbol{a}) = \sum_{i=1}^{N} 2^{i-1} \theta_i(\boldsymbol{a}). \tag{5}$$

The indicator θ_i is one when denominator i is present and zero otherwise, and S is an integer that identifies this set of denominators. Integrals with different powers of the same denominators belong to the same sector because S depends only on which indices are positive. Removing one or more denominators gives a subsector containing a proper subset of the original sector's denominators.

Scaleless sectors contain no nonzero mass or kinematic scale and therefore vanish in dimensional regularization, while changes of loop integration variables can yield symmetry relations between integrals. Kira uses these zero sectors and symmetry mappings to reduce the number of unknowns [2]. We can then use the sector and index labels to write the identities that relate the remaining family members.

3 From integration to algebraic reduction

To express the targets in terms of master integrals, we generate relations between family members through integration by parts and solve the resulting equations. Evaluating the master integrals then gives the target integral values needed to calculate the amplitude.

3.1 How integration by parts produces relations

For differentiable functions $f(z)$ and $g(z)$ of a real variable z, the product rule gives the integration identity between endpoints z_0, z_1:

$$\int_{z_0}^{z_1} f'(z)g(z)\,\mathrm{d}z = [f(z)g(z)]_{z_0}^{z_1} - \int_{z_0}^{z_1} f(z)g'(z)\,\mathrm{d}z. \tag{6}$$

Primes denote derivatives with respect to z, and the brackets denote the endpoint difference $f(z_1)g(z_1) - f(z_0)g(z_0)$. To apply this product rule to loop-momentum derivatives, we derive the integration-by-parts (IBP) identity where the regulated integral has a vanishing boundary term and then extend it by analytic continuation [4, 3]. For a loop momentum ℓ_k and a vector $v \in \{\ell_1, \ldots, \ell_L, p_1, \ldots, p_E\}$, the identity is

$$0 = \int_\ell \frac{\partial}{\partial \ell_k^\mu} \left(v^\mu \prod_{i=1}^{N} D_i^{-a_i} \right). \tag{7}$$

The index μ is a spacetime index, and a repeated Lorentz index means that we sum over components using the spacetime metric. The derivative acts on both v^μ and the factors $D_i^{-a_i}$, so the product rule gives

$$0 = \int_\ell \left[\frac{\partial v^\mu}{\partial \ell_k^\mu} - \sum_{i=1}^{N} a_i \frac{v^\mu (\partial D_i / \partial \ell_k^\mu)}{D_i} \right] \prod_{j=1}^{N} D_j^{-a_j}. \tag{8}$$

The first term equals d for $v = \ell_k$ and vanishes when v is an external momentum or another independent loop momentum. Differentiating an inverse propagator produces scalar products, which we express through the family basis to convert each term into an integral with shifted indices:

$$\sum_{j=1}^{n_\lambda} c_{\lambda j}(\boldsymbol{a}; \boldsymbol{x}, \epsilon) I_{\boldsymbol{a}+\boldsymbol{\Delta}_{\lambda j}}(\boldsymbol{x}, \epsilon) = 0. \tag{9}$$

Here λ identifies the chosen pair (k, v), n_λ is the number of terms, and $\boldsymbol{\Delta}_{\lambda j}$ is the integer change in the index vector for term j. Differentiation gives the coefficient $c_{\lambda j}$, which depends on the integral indices and the parameters $\boldsymbol{x}, \epsilon$ [4].

The L choices of differentiated momentum and $L+E$ choices of v give $L(L+E)$ basic identities at each index vector, some of which may depend linearly on the others. Further relations follow from Lorentz invariance because a common rotation or Lorentz boost of all external momenta preserves a scalar integral, giving the infinitesimal identity

$$\sum_{\beta=1}^{E} \left(p_\beta^\nu \frac{\partial}{\partial p_{\beta\mu}} - p_\beta^\mu \frac{\partial}{\partial p_{\beta\nu}} \right) I_{\boldsymbol{a}} = 0. \tag{10}$$

Here μ, ν are free spacetime indices, meaning that the identity holds for each pair of components, and the derivatives act on the external-momentum dependence of the integral. Contracting these indices with external momenta gives scalar relations [5] that join the IBP and symmetry identities to form the reduction system.

3.2 From relations to master integrals

Let n_I be the number of distinct integrals in the generated equations and n_{eq} the number of equations. We arrange these integrals in a column vector so that the system takes the matrix form

$$A(\boldsymbol{x}, \epsilon)\, \boldsymbol{I} = 0, \qquad \boldsymbol{I} = (I_{\boldsymbol{a}^{(1)}}, \ldots, I_{\boldsymbol{a}^{(n_I)}})^{\mathsf{T}}. \tag{11}$$

The superscript T denotes transpose, so $\boldsymbol{I}$ is a column vector of integrals whose dependence on $\boldsymbol{x}, \epsilon$ we leave implicit here. The matrix A has n_{eq} rows and n_I columns, with each entry A_{ij} giving the coefficient of integral j in equation i. Solving this system expresses dependent integrals in terms of an independent set. This set is a master basis if the reduction relations express every required integral as a linear combination of its members. For the standard class of Feynman integrals at fixed graph, this basis contains a finite number of masters [6]. We write them as $M_1, \ldots, M_{n_M}$, where n_M is the number of master integrals, so the reduction formula is

$$I_{\boldsymbol{a}}(\boldsymbol{x}, \epsilon) = \sum_{m=1}^{n_M} C_{\boldsymbol{a}m}(\boldsymbol{x}, \epsilon) M_m(\boldsymbol{x}, \epsilon). \tag{12}$$

Here $C_{\boldsymbol{a}m}$ is the coefficient of master M_m in the reduction of target $I_{\boldsymbol{a}}$. The solver obtains it by adding, multiplying and dividing the identity coefficients, so it is a rational function, meaning a ratio of polynomials in $\boldsymbol{x}, \epsilon$. Different independent sets can define different master bases, and an insufficient equation set can leave additional integrals unresolved. Establishing the master basis therefore requires checking that the generated equations are enough to reduce the required integrals.

Equation (12) gives the coefficients multiplying the masters, whose evaluation requires a further calculation, for example by solving differential equations with known values or limits as boundary conditions [5]. Kira provides the reduction rules needed for this calculation by generating and processing the identities through the procedure described next.

4 How Kira performs the reduction

Kira implements the Laporta approach by generating a finite set of identities and eliminating integrals according to an ordering [7]. To obtain the coefficients in Eq. (12), Kira selects seeds, removes redundant equations and solves the retained system. We examine how Kira performs these steps to specify the seed and elimination choices that the proposed models would make.

4.1 Kira's input and seed generation

The user supplies the families in Eq. (3), relations among the parameters $\boldsymbol{x}$, the target list, and bounds on equation generation so that Kira can initialize the families and sectors. During this initialization, Kira detects scaleless sectors and identifies symmetry relations and mappings between compatible families [2].

A seed specifies the index vector $\boldsymbol{a}$ at which Kira evaluates Eq. (9) to generate several derivative identities, and the seed set may include integrals beyond the target list. The shifts $\boldsymbol{\Delta}_{\lambda j}$ in these

identities can introduce further integrals outside the seed set, so the resulting system depends on both the chosen seeds and the terms generated from them. To set limits on the indices of the seeds, Kira uses the quantities

$$t(\boldsymbol{a}) = \sum_{i=1}^{N} \theta_i(\boldsymbol{a}), \qquad r(\boldsymbol{a}) = \sum_{a_i>0} a_i, \tag{13}$$

$$s(\boldsymbol{a}) = -\sum_{a_i<0} a_i, \qquad n_{\mathrm{dot}}(\boldsymbol{a}) = r(\boldsymbol{a}) - t(\boldsymbol{a}). \tag{14}$$

Here t is the number of distinct denominators, r is the sum of their powers, and s is the sum of the numerator powers. The quantity n_{dot} is the sum of the extra denominator powers beyond one for each denominator. We use this symbol to keep d reserved for the integration dimension, and we restrict equation generation by specifying the allowed sectors and limits

$$r \le r_{\max}, \qquad s \le s_{\max}, \qquad n_{\mathrm{dot}} \le n_{\mathrm{dot,max}}, \tag{15}$$

where $r_{\max}$, $s_{\max}$ and $n_{\mathrm{dot,max}}$ are the upper limits on the corresponding quantities. The user sets $r_{\max}$ and $s_{\max}$ and may also set $n_{\mathrm{dot,max}}$. Because these choices determine which seeds contribute equations, a reduction run must establish whether the bounds supply enough relations for the targets [8].

Kira 3 changes seeding in symmetry-related sectors and provides the optional `truncate_sp` setting to restrict numerator seeding in subsectors, with parameter choices that require checks against the expected master basis [9]. The generated system can still contain redundant equations, which elimination identifies as it processes the relations into reduction rules.

4.2 Integral ordering, sector order and forward elimination

Elimination solves an identity for one integral and substitutes the result into other identities, using an integral ordering to select which unknown to eliminate from each equation. Kira defines these orderings through sector labels and the powers of the numerator and denominator factors, with choices that prioritize numerator powers or repeated denominators and thereby affect the master basis and runtime [8].

To describe this procedure, write $I_j = I_{\boldsymbol{a}^{(j)}}$ for the integrals in Eq. (11) and choose the ordering $I_1 \succ I_2 \succ \cdots \succ I_{n_I}$, where $\succ$ gives priority for elimination. In a row whose first nonzero term contains I_k, let b_j denote the coefficient of I_j, so choosing the nonzero coefficient b_k as the pivot gives

$$b_k I_k + \sum_{j>k} b_j I_j = 0 \quad \Longrightarrow \quad I_k = -\sum_{j>k} \frac{b_j}{b_k} I_j. \tag{16}$$

The coefficients b_j depend on $\boldsymbol{x}, \epsilon$, and substituting this expression into the other rows removes their dependence on I_k. Repeating this step gives forward elimination, which builds a triangular system in which each pivot integral depends on integrals later in the ordering. Back substitution then replaces the remaining pivot integrals using their own rules, completing the Laporta solution [7].

Sector scheduling sets the order in which the solver processes groups of integrals or equations, taking into account which other integrals their equations contain. Ordinary IBP identities for a sector involve that sector and its subsectors because nonpositive indices remain nonpositive when applying the derivative. Equations for one sector can therefore depend on integrals from its subsectors. The Kira 3 description starts equation generation with sectors earlier in its ordering and inserts previously obtained pivot relations into new equations [9], with the order of processing depending on the stage of the reduction.

When several rows share the same first integral, the choice of pivot row changes the substitutions that follow. Choosing row ρ to eliminate column k, for example, updates row i by

$$A_{ij} \longleftarrow A_{ij} - \frac{A_{ik}}{A_{\rho k}} A_{\rho j}, \qquad A_{\rho k} \neq 0. \tag{17}$$

Here i is the index of a row other than the pivot row, so $i \neq \rho$, and j is the column index. Subtracting the stated multiple of row ρ sets the coefficient in column k to zero and can create nonzero entries in other columns, a change called fill-in when a previously zero entry becomes nonzero. These substitutions also make the coefficient expressions longer by combining their numerators and denominators, so elimination can require more time and memory as it proceeds. Redundant equations add to this work [10], so Kira uses modular elimination to select equations before processing their full symbolic coefficients.

4.3 Removing redundant equations and selecting target dependencies

To reduce the number of equations it must solve, Kira identifies rows that follow from other equations. Elimination reduces every coefficient in such a row to zero, allowing the solver to omit that row. Kira tests these dependencies with modular arithmetic [2] by substituting sample values $\boldsymbol{x}_0, \epsilon_0$ and mapping the coefficients to a finite field $\mathbb{F}_P$ for a prime integer P:

$$A(\boldsymbol{x}, \epsilon) \longmapsto A(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{18}$$

Integers from 0 to $P - 1$ represent the elements of $\mathbb{F}_P$, and addition and multiplication keep results in this range by taking the remainder upon division by P. Division uses the modular inverse of a nonzero element, which multiplies that element to one modulo P, so the sampling procedure excludes points where a required denominator vanishes. This arithmetic keeps field elements bounded in size while allowing the solver to identify independent rows and retain their original symbolic identities.

The matrix rank is the largest number of independent rows, where no row is a linear combination of the others in the chosen set. Some sample values can make otherwise independent rows dependent, giving a rank smaller than that of the symbolic matrix. Checks must account for this possibility when using the probabilistic equation-selection method developed by Kant [10].

Kira also tracks which independent equations contribute to the target reductions, using the dependencies of the forward-eliminated system for this selection in Kira 3 [9]. Seed selection therefore supplies the integrals from which Kira generates equations, and equation selection then retains the relations needed for the targets so that the solver can perform the full reduction on this smaller system.

4.4 Back substitution and the final symbolic result

To obtain $C_{\boldsymbol{a}m}$ from the retained equations, Kira applies back substitution to the pivot rules produced by forward elimination. Since these rules can contain other pivot integrals, back substitution inserts their expressions recursively until the right-hand sides contain only masters, giving Eq. (12). Kira's original symbolic solver carries out this calculation using rational-function arithmetic through the computer algebra system Fermat [2].

Kira also solves systems at finite-field sample points and reconstructs the rational coefficients with FireFly, the reconstruction library introduced into the solver in Kira 2.0 [11]. In this procedure, each sampled coefficient has the form

$$C_{\boldsymbol{a}m}(\boldsymbol{x}_0, \epsilon_0) \bmod P. \tag{19}$$

The modular operations use exact arithmetic, and reconstruction recovers $C_{\boldsymbol{a}m}(\boldsymbol{x}, \epsilon)$ from sufficient samples before checking the result at additional points. Reconstruction therefore forms part of the calculation of the symbolic reduction coefficients, after which the master integrals still require evaluation [11].

Because seed selection determines the equations that enter this procedure and elimination decisions determine the substitutions and intermediate expressions, the proposed ML models would guide these two operations to reduce the time and memory needed for the reduction.

5 ML for seed selection and elimination ordering

To reduce the time and memory needed for IBP reduction, we will study two System 1 models: one for seed selection and one for elimination ordering. Here a System 1 model is a small machine learning model that learns from recorded reductions and makes a choice from the information available at a given stage. We would add the seed model before equation generation and the elimination model at the stage where the solver selects a pivot. Each model would assign scores to the available choices, and Kira would perform the algebra for the selected choice. Training these models would require target integrals with known master expansions, the seeds that supply their reduction equations, and the steps used to solve those equations.

5.1 Selecting seeds before equation generation

To select seeds, the first model would receive the family definition, the target index vectors $\boldsymbol{a}$, the chosen master basis and a set of candidate seeds within specified sectors and index bounds. For each candidate, it would use the seed indices and the target information to estimate how useful the resulting equations would be for the requested reduction.

During training, we would pair each target with the seed sets recorded in its known reductions. These examples would teach the model to rank seeds that together provide enough equations to reduce the target. Different reductions can use different sufficient seed sets, so the training data would retain these alternatives and their equation counts. When applying the trained model, we would select highly ranked seeds and ask Kira to generate their identities. If the equations leave a target unresolved in the chosen master basis, we would add further seeds and repeat the reduction. The second model would guide the elimination of the resulting equations.

5.2 Choosing elimination steps within the solver

To limit fill-in and the growth of coefficient expressions, the second model would rank the allowed pivot choices using the current equations, their nonzero coefficients and the integral ordering. For each recorded reduction step, the training example would contain the equations before the choice, the available pivots, the selected row and integral, and the operations that followed. The remaining recorded steps and their operation counts would show how that choice contributed to a complete reduction.

During a reduction, the model would select an allowed pivot and Kira would carry out the row operations in Eq. (17). The model would then receive the updated system and select the next pivot, continuing until Kira can complete the reduction by back substitution. Each choice must respect the integral ordering and chosen master basis. Preparing the model inputs and obtaining its scores must take less time than its choices save in the algebraic calculation. Kira would use its existing reconstruction procedure for calculations over finite fields.

5.3 Integral data required for training and testing

To train and test these models, we would require examples that specify the family denominators, kinematic parameters, target integrals and master basis, together with the exact coefficients in Eq. (12). Each example would also contain its source IBP equations, their seeds and derivative operators, and any symmetry relations used. These records would connect the seed model's choices to the equations available to the elimination model.

We would record both the operations used to construct each target and the elimination and back-substitution steps used to reduce it. The records would include intermediate equations, pivot choices and operation counts, so the elimination model can learn from complete sequences. The stored master coefficients would supply the reference answer for checking that each target reduces completely in the chosen basis, a requirement discussed in Section 3 and the Kira documentation [9]. During tests, the models would receive the problem information available at their respective stages; the recorded solutions would remain separate for checking the results. Section 6 explains how we would construct the target integrals and obtain these records together.

6 Target Integral Generation for Machine Learning

To train and test the two models described in Section 5, we would generate target integrals and record the seeds and elimination steps used to reduce each target. We would start from known master integrals, form linear combinations and use IBP identities to obtain further members of the same family. Repeating this construction would produce targets with known master expansions and recorded relations back to the starting basis. We would then use these relations to obtain the reduction sequences required for training.

6.1 Integral families and starting master bases

To generate examples for different types of Feynman integrals, we would select families with different loop orders, propagator structures, masses and external kinematics, using Eq. (3) to define each family. For each family, we would use a master basis $M_1, \ldots, M_{n_M}$ established by known reduction relations or a checked reference reduction [2, 9]. We would keep the master integrals symbolic, since their numerical values are unnecessary for constructing algebraic relations. We would start with each master expressed as itself. If $\boldsymbol{\mu}^{(m)}$ is the index vector of master M_m, this initial record is

$$M_m = I_{\boldsymbol{\mu}^{(m)}}, \qquad C_{\boldsymbol{\mu}^{(m)}n} = \delta_{mn}, \qquad m, n = 1, \ldots, n_M. \tag{20}$$

Here δ_{mn} is one when $m = n$ and zero otherwise, so each master has coefficient one for itself and zero for the other masters. These records provide the starting coefficients from which later constructions obtain their exact master expansions.

For each family, we would store the denominators D_i, the kinematic parameters $\boldsymbol{x}$, the dimension $d = 4 - 2\epsilon$, the sector definitions and the chosen master basis. Keeping these definitions fixed during each construction would ensure that every integral index vector refers to a specified integral. To include different loop orders or propagator structures, we would repeat the construction in separately defined families.

6.2 Linear combinations with known master expansions

To construct target expressions with known master expansions, we would combine family integrals whose reductions we have already obtained. The initial choices would be the masters

themselves, and later choices would also include integrals generated through the identities described below. Let $I_{\boldsymbol{a}^{(1)}}, \ldots, I_{\boldsymbol{a}^{(n_T)}}$ be the selected integrals, where n_T is their number. Choosing weights $w_j(\boldsymbol{x}, \epsilon)$ gives a target expression $\mathcal{T}$ whose master expansion follows by substitution. Suppressing the common arguments $\boldsymbol{x}, \epsilon$, we obtain

$$\mathcal{T} = \sum_{j=1}^{n_T} w_j I_{\boldsymbol{a}^{(j)}} = \sum_{m=1}^{n_M} C_m^{\mathcal{T}} M_m, \qquad C_m^{\mathcal{T}} = \sum_{j=1}^{n_T} w_j C_{\boldsymbol{a}^{(j)}m}. \tag{21}$$

Here j identifies a selected integral, m identifies a master, and $C_m^{\mathcal{T}}$ is the coefficient of that master in $\mathcal{T}$. The coefficients $C_{\boldsymbol{a}^{(j)}m}$ are the known reductions defined in Eq. (12), so multiplication and addition give the exact answer for the new expression.

The weights may be constants or rational functions of the masses, external momentum invariants and ϵ. They remain independent of the loop momenta, and we would choose their mass dimensions so that the terms in the sum have the same mass dimension. This dependence is consistent with the coefficient functions in IBP reduction [9]. Varying the selected integrals and their weights would produce different target expressions, including expressions with cancellations between master coefficients. The corresponding seed and elimination records would include the reductions of the selected integrals and the operations used to combine their results.

Equation (21) defines a linear combination of family integrals. Identifying that expression with one integral $I_{\boldsymbol{b}}$ requires an identity involving the specific index vector $\boldsymbol{b}$. We would therefore use IBP relations to determine combinations that correspond to individual family members, as described next. This step supplies targets in their integral form and a separate record of their known master expansions.

6.3 Constructing individual targets through IBP identities

To construct a new individual target with a known reduction, we would use IBP identities to express that integral in terms of integrals whose reductions are already known. We would generate the identities in Eq. (9) from a seed vector $\boldsymbol{\sigma}$ and record the differentiated loop momentum and the vector v used in Eq. (7). The pair (k, v) specifies the IBP operator, so the seed and this pair determine the source of each equation [2]. Consider an identity that contains a new integral $I_{\boldsymbol{b}}$ and n_R integrals whose reductions we already know:

$$h_0 I_{\boldsymbol{b}} + \sum_{j=1}^{n_R} h_j I_{\boldsymbol{a}^{(j)}} = 0, \qquad h_0 \neq 0. \tag{22}$$

Here n_R is the number of known integral terms, and h_0, h_j are the coefficients obtained from the identity, with their dependence on $\boldsymbol{x}, \epsilon$ implicit. Solving for $I_{\boldsymbol{b}}$ and substituting the known master expansions gives

$$\begin{aligned} I_{\boldsymbol{b}} &= -\sum_{j=1}^{n_R} \frac{h_j}{h_0} I_{\boldsymbol{a}^{(j)}}, \\ C_{\boldsymbol{b}m} &= -\sum_{j=1}^{n_R} \frac{h_j}{h_0} C_{\boldsymbol{a}^{(j)}m}. \end{aligned} \tag{23}$$

The identity therefore determines both the new integral and its coefficients in the master basis. We would add this integral to the collection of known reductions and use it in later constructions, retaining the conditions under which division by h_0 is valid. Starting with the coefficients in Eq. (20) and applying Eq. (23) repeatedly would build each target from previously known integrals. To recover its reduction, we would follow these recorded relations from the target back to the masters and substitute the corresponding expressions.

Some identities contain several integrals whose reductions are still unknown. In that case, we would combine enough relations to solve for those integrals before extending the collection, retaining a target only when these relations determine its complete master expansion. This may require solving a coupled system before any of its new integrals can enter the collection. Changing the seeds and IBP operators would generate targets in different sectors and with different numerator and denominator powers. We would record the quantities $t, r, s, n_{\mathrm{dot}}$ from Eq. (14) to describe this variation. Each accepted construction must give a reduction entirely in the chosen master basis, with the exact coefficients retained as its reference answer.

6.4 Records for seed selection and elimination

To train the models on the choices made during reduction, we would store the source equations and the operations used to reduce each target. Each record would include the family, the target index vector or target expression, and its exact master coefficients. For every source IBP equation, we would store the seed $\boldsymbol{\sigma}$, the operator (k, v) and the original equation before substitutions. If a construction also uses a symmetry relation, we would record that relation and its integral mapping. These records would identify the seeds and identities needed to reproduce each equation used in the construction.

The seed record of a new target must include the seeds used for the integrals that appear in its construction. For the single-identity step in Eq. (22), let $\mathcal{S}_{\boldsymbol{a}}$ be the stored seed set for integral $I_{\boldsymbol{a}}$. The new record is then

$$\mathcal{S}_{\boldsymbol{b}} = \{\boldsymbol{\sigma}\} \cup \bigcup_{j=1}^{n_R} \mathcal{S}_{\boldsymbol{a}^{(j)}}, \qquad \mathcal{S}_{\boldsymbol{\mu}^{(m)}} = \varnothing. \tag{24}$$

The union collects the seed of the new identity and the seeds used in all its earlier reductions, keeping each seed once. The empty set $\varnothing$ applies to a starting master because expressing it as itself requires no reduction equations. When a construction uses several new identities, we would include every seed used to generate them. We would also retain the individual equations and their dependencies, since the seed set alone does not specify the reduction sequence.

We would also store the candidate seeds considered during construction and the sector and index bounds used to generate them. The collected seeds would provide a set known to supply enough equations for each accepted target. If different constructions produce the same target, we would retain their seed sets together with the corresponding reduction records. For a target expression $\mathcal{T}$, we would combine the seed records of its constituent integrals to obtain an initial sufficient set, which cancellations in the combined expression may allow us to reduce further.

For the elimination model, we would replay the source equations in an allowed forward reduction order and record the system before each decision. The record would contain the available pivot choices, the selected equation, the integral eliminated, and the row operations and substitutions that follow. We would require these steps to respect the chosen integral ordering and master basis, using the elimination and back-substitution operations described in Section 4 [7]. The target's final master expansion would remain the reference answer for checking the complete sequence. This replay would turn the construction record into examples of the decisions that the model will encounter inside the solver.

We would record the operations used to construct expressions and the operations used to reduce the source equations separately. For each reduction sequence, we would state how we count additions, subtractions, multiplications and divisions of coefficients, and record changes in the number of nonzero matrix entries. Each example would therefore include the reduction sequence, its operation counts and its exact result. Constructing a target in this way gives a known reduction without requiring a search through every possible reduction sequence. We would

identify a seed set or sequence as minimal only when a mathematical argument establishes that it uses the fewest seeds or operations under the stated rules.

6.5 Generated data for training and tests

To test the models on generated targets excluded from training, we would divide the constructed examples into separate training and test collections and keep the test solutions hidden from the models. Targets related by symmetry, integral relabeling or different construction histories would remain in the same collection, so a stored version of a test target cannot also become a training example. We would reserve complete families to measure performance on families absent from training, and include generated targets with larger powers and higher loop orders to examine performance on larger reductions.

The seed model would learn from the recorded sufficient seed sets, and the elimination model would learn from the recorded equations and reduction steps. During tests, each model would receive only the corresponding problem information, and the exact master coefficients would provide the reference for checking its result. For examples with an established minimum, the recorded operation counts would allow a comparison with that minimum. The remaining examples would provide known reductions against which to compare the model's choices.

Before including an example in either collection, we would check its identities and reproduce its master expansion from the stored source equations. Keeping the original equations and their construction history would make these checks possible for both individual integrals and linear combinations of integrals.

Citations

[1] Stefan Weinzierl. Introduction to Feynman integrals, 2010. arXiv:1005.1855.

[2] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[3] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers*. UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[4] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[5] Thomas Gehrmann and Ettore Remiddi. Differential equations for two-loop four-point functions. *Nuclear Physics B*, 580:485–518, 2000. arXiv:hep-ph/9912329.

[6] A. V. Smirnov and A. V. Petukhov. The number of master integrals is finite, 2010. arXiv:1004.4199.

[7] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[8] Philipp Maierhöfer and Johann Usovitsch. Kira 1.2 release notes, 2018. arXiv:1812.01491.

[9] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection, 2025. arXiv:2505.20197.

[10] Philipp Kant. Finding linear dependencies in integration-by-parts equations: A Monte Carlo approach. *Computer Physics Communications*, 185:1473–1476, 2014. arXiv:1309.7287.

[11] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.